

Relatório — Sprint 3 · Estruturas de Dados

ChargeGrid Intelligence · NexusCharge — EV Challenge 2026 · FIAP + GoodWe

Programa analisado: `gestao_sesoes.py`

Execução:

```
python gestao_sesoes.py          # menu interativo
python gestao_sesoes.py --autoteste # verificação dos algoritmos
```

Todas as medições deste relatório foram produzidas pela opção **6 — Comparar algoritmos** do próprio programa, e podem ser reproduzidas.

1. Estrutura utilizada para representar uma sessão

A sessão é uma **classe** (`Sessao`), declarada como `dataclass`:

```
@dataclass
class Sessao:
    id: int
    energia: float
    tempo: float
    custo: float
    veiculo: str = "-"
    codigo: str = ""
    tarifa: float = 0.0
    conector: str = "-"
    tipo: str = "P"
    data: str = ""
    hora: int = 0
    status: str = "ENCERRADA"
```

Por que classe e não dicionário

Um dicionário aceitaria `s["engria"]` — repare no erro de digitação — sem reclamar, e só quebraria na hora de somar, com um `KeyError` longe da origem do problema. A classe fixa o contrato: os campos existem, têm tipo declarado, e o erro de digitação aparece na escrita. Como as funções de busca e ordenação leem sempre os mesmos atributos, essa garantia vale mais aqui do que a flexibilidade do dicionário.

O `@dataclass` gera `__init__` e `__repr__` a partir da declaração dos campos — é a mesma classe do exemplo do enunciado, sem o construtor repetitivo.

Atributos além dos quatro obrigatórios

Os quatro exigidos são `id`, `energia`, `tempo` e `custo`. Os demais existem porque o programa consome o histórico real do sistema:

Atributo	Papel
<code>codigo</code>	identificador da sessão no app web (ex.: <code>CGI-1480B94E</code>)
<code>veiculo</code>	placa, exibida na listagem e na consulta
<code>tarifa</code>	R\$/kWh aplicada, permite conferir <code>custo ÷ energia</code>
<code>conector</code>	carregador usado (<code>P1-C3</code>), liga a sessão ao posto
<code>tipo</code>	categoria tarifária (P/A/C)
<code>data</code> , <code>hora</code>	quando a recarga começou
<code>status</code>	<code>PAGA</code> (veio do banco) ou <code>MANUAL</code> (cadastrada no menu)

Armazenamento: uma lista

```
sessoes: List[Sessao] = []
sessoes.append(nova_sessao)
len(sessoes)
```

A escolha da `list` é deliberada e vale registrar: um `dict` indexado por ID resolveria a busca em $O(1)$ e tornaria a Sprint inteira sem assunto. A lista tem posições, e é a existência de posições que dá sentido a ordenar e a buscar por divisão do intervalo. O custo dessa escolha é exatamente o que a seção 5 mede.

2. Funcionamento geral do programa

Origem dos dados

Ao iniciar, o programa tenta ler as sessões pagas de `chargegrid.db` — o mesmo banco que o app web (`app.py`) alimenta quando uma recarga é encerrada e paga. Cada linha vira um objeto `Sessao` e recebe um `id` inteiro sequencial.

Sem banco, sem tabela ou sem sessões, o programa abre com a lista vazia em vez de recusar-se a iniciar; o cadastro pelo menu preenche a lista. O CLI **nunca escreve** no banco: é ferramenta de análise, e manter a escrita fora daqui impede que um experimento de ordenação altere o histórico de faturamento.

Menu

```
=====
                ESTAÇÃO DE RECARGA
=====

1 - Nova sessão de recarga
2 - Listar sessões
3 - Buscar sessão
4 - Ordenar sessões
5 - Estatísticas
6 - Comparar algoritmos (Big-0 na prática)
7 - Encerrar
```

O laço é `while True` e só termina na opção 7. Opção inexistente é avisada e o menu volta a ser exibido.

Divisão em funções

Função	Responsabilidade
<code>carregar_do_banco()</code>	lê o histórico e monta a lista inicial
<code>cadastrar_sessao()</code>	coleta e valida os dados de uma nova sessão
<code>listar_sesoes()</code>	imprime a tabela na ordem atual da lista
<code>buscar_sessao()</code>	pergunta o algoritmo, executa e mostra o resultado
<code>ordenar_sesoes()</code>	pergunta critério e algoritmo, ordena, relista
<code>mostrar_estatisticas()</code>	imprime o resumo agregado
<code>comparar_algoritmos()</code>	mede comparações em entradas crescentes
<code>busca_sequencial()</code> <code>busca_binaria()</code>	algoritmos de busca
<code>bubble_sort()</code> <code>insertion_sort()</code>	algoritmos de ordenação
<code>estatisticas()</code>	agrega em uma passada e devolve os valores
<code>ler_numero()</code> <code>ler_texto()</code>	entrada validada
<code>main()</code>	laço do menu e despacho das operações

As funções de algoritmo são puras em relação ao menu: recebem a lista e devolvem resultado, sem imprimir nada. Isso é o que permite chamá-las dentro do comparativo da opção 6 e do autoteste sem duplicar código.

Critérios de ordenação

O critério é passado como parâmetro — uma função que extrai o valor comparável:

```
CRITERIOS = {
    "1": ("ID", lambda s: s.id),
    "2": ("Energia consumida", lambda s: s.energia),
    "3": ("Custo da sessão", lambda s: s.custo),
    "4": ("Tempo de recarga", lambda s: s.tempo),
}
```

Sem isso seriam quatro cópias de cada algoritmo, uma por atributo. A lógica de ordenação fica num lugar só e os quatro critérios são configuração.

Estatísticas

Calculadas sobre as sessões realmente armazenadas, em **uma única passada** pela lista — quatro agregações independentes seriam quatro varreduras, e todas precisam do mesmo elemento ao mesmo tempo:

```
===== ESTATÍSTICAS DA ESTAÇÃO =====

Sessões realizadas .... 180
Energia fornecida ..... 2878.19 kWh
Faturamento ..... R$ 4205.76
Ticket médio ..... R$ 23.37
Maior consumo ..... 38.41 kWh
Menor consumo ..... 2.07 kWh
Tempo total ..... 18445.0 min
```

Lista vazia devolve zeros em vez de estourar: `min()` sobre lista vazia é `ValueError`, e o menu precisa continuar de pé.

Validações

Situação	Tratamento
Texto onde se espera número	<code>try/except ValueError</code> , nova tentativa (até 3)
Energia, tempo ou custo negativo	recusado com a faixa aceita na mensagem
ID duplicado	verificado com <code>busca_sequencial</code> antes de inserir
Opção de menu inválida	avisada; o laço continua
Enter vazio	cancela a operação e volta ao menu
<code>Ctrl+C</code> / fim de entrada	encerra a operação sem <i>traceback</i>
Exceção inesperada numa operação	capturada em <code>main()</code> ; o menu sobrevive

Nenhuma entrada do usuário encerra o programa inesperadamente.

3. Algoritmo de busca utilizado

Dois algoritmos, escolhidos pelo usuário na opção 3.

3.1 Busca sequencial

```
def busca_sequencial(colecao, id_procurado):
    comparacoes = 0
    for i in range(len(colecao)):
        comparacoes += 1
        if colecao[i].id == id_procurado:
            return i, comparacoes
    return -1, comparacoes
```

Percorre a lista da primeira posição à última e devolve o índice do elemento, ou **-1** se não existir. **Não exige lista ordenada** — é a única busca aplicável logo depois de um cadastro, antes de qualquer ordenação. É também a que `id_existe()` usa para barrar ID duplicado.

3.2 Busca binária

```
def busca_binaria(colecao, id_procurado):
    inicio, fim, comparacoes = 0, len(colecao) - 1, 0
    while inicio <= fim:
        meio = (inicio + fim) // 2
        comparacoes += 1
        if colecao[meio].id == id_procurado:
            return meio, comparacoes
        if colecao[meio].id < id_procurado:
            inicio = meio + 1      # descarta a metade de baixo
        else:
            fim = meio - 1        # descarta a metade de cima
    return -1, comparacoes
```

Compara com o elemento central e descarta metade do intervalo a cada passo.

Pré-condição: a lista precisa estar ordenada pelo mesmo atributo da busca. Sobre uma lista desordenada o algoritmo descarta a metade errada e devolve **-1** para um elemento que existe. Quem chama é responsável por garantir isso — por esse motivo `buscar_sessao()` ordena por ID com o `insertion_sort` antes de chamar, e informa na tela:

```
Lista ordenada por ID primeiro (179 comparações no insertion sort).
Busca binária: encontrada na posição 41 (6 comparações).
```

Nenhuma função pronta (`index`, `in`, `bisect`) substitui os dois algoritmos.

4. Algoritmo de ordenação utilizado

Dois algoritmos, escolhidos pelo usuário na opção 4. Ambos ordenam a lista *in-place* e devolvem o número de comparações realizadas.

4.1 Bubble Sort

```
def bubble_sort(colecao, chave):
    n = len(colecao)
    comparacoes = 0
    for i in range(n):
        for j in range(n - 1 - i):
            comparacoes += 1
            if chave(colecao[j]) > chave(colecao[j + 1]):
                colecao[j], colecao[j + 1] = colecao[j + 1], colecao[j]
    return comparacoes
```

Compara pares vizinhos e troca os que estão fora de ordem. A cada passada do laço externo o maior elemento restante "flutua" até o fim, e por isso o laço interno encolhe (`n - 1 - i`).

Não tem parada antecipada: mesmo com a lista já ordenada, faz o mesmo número de comparações.

4.2 Insertion Sort

```
def insertion_sort(colecao, chave):
    comparacoes = 0
    for i in range(1, len(colecao)):
        atual = colecao[i]
        j = i - 1
        while j >= 0:
            comparacoes += 1
            if chave(colecao[j]) <= chave(atual):
                break
            colecao[j + 1] = colecao[j]
            j -= 1
        colecao[j + 1] = atual
    return comparacoes
```

Trata a porção à esquerda como já ordenada e insere cada novo elemento na posição certa dela, deslocando os maiores para a direita. O `break` é o que dá a ele o melhor caso linear: encontrando um vizinho menor ou igual, para de recuar.

Nem `sort()` nem `sorted()` são usados em nenhum ponto do arquivo.

5. Análise de complexidade (Big-O)

5.1 Busca sequencial — $O(n)$

O trecho que provoca o crescimento:

```
for i in range(len(colecao)):
```

O laço visita uma posição por iteração.

- **melhor caso:** elemento na primeira posição → 1 comparação
- **pior caso:** elemento na última, ou ausente → n comparações
- **caso médio:** $\sim n/2$ comparações

A notação descreve o pior caso e descarta a constante: $n/2$ e n crescem na mesma proporção, então ambos são $O(n)$. Dobrar o número de sessões dobra o número máximo de verificações.

5.2 Busca binária — $O(\log n)$

```
while inicio <= fim:
    meio = (inicio + fim) // 2
```

Cada iteração descarta metade do intervalo restante. Partindo de n posições, a sequência $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$ chega a 1 depois de $\log_2(n)$ passos. Medido pelo programa (elemento ausente, pior caso):

n	Sequencial	Binária	$\log_2(n)$
10	10	4	3,3
100	100	7	6,6
1.000	1.000	10	10,0
10.000	10.000	14	13,3

Dobrar a entrada acrescenta **uma** comparação à busca binária, contra o dobro na sequencial.

5.3 Bubble Sort — $O(n^2)$

Os dois laços aninhados:

```
for i in range(n):
    for j in range(n - 1 - i):
```

O externo roda n vezes; o interno roda $n-1-i$ vezes, encolhendo a cada passada. O total é a soma

$$(n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 = (n^2 - n)/2$$

Na notação assintótica a constante $\frac{1}{2}$ e o termo linear $-n$ são descartados: sobra n^2 . A medição do programa bate com a fórmula exatamente, porque não há parada antecipada:

n	Bubble (medido)	$n(n-1)/2$
10	45	45
25	300	300
50	1.225	1.225
100	4.950	4.950
200	19.900	19.900

De $n=100$ para $n=200$ as comparações vão de 4.950 para 19.900: o dobro de dados, **quatro vezes** o trabalho. É a assinatura do crescimento quadrático.

Sobre o histórico real do projeto, com 180 sessões, ordenar por energia custou **16.110 comparações** — exatamente $180 \times 179 / 2$.

5.4 Insertion Sort — $O(n^2)$ no pior caso, $O(n)$ no melhor

```
for i in range(1, len(colecao)):
    while j >= 0:
```

No pior caso (lista em ordem inversa) o `while` recua até o início a cada elemento, e o total volta a ser $n(n-1)/2$ — **mesma classe do bubble**.

No melhor caso (lista já ordenada) a condição falha na primeira comparação de cada elemento: **$n-1$** comparações no total, crescimento **linear**.

Medido pelo programa, sobre a mesma lista ordenada duas vezes seguidas:

n	Bubble (2ª vez)	Insertion (2ª vez)
50	1.225	49
200	19.900	199

O bubble não muda nada; o insertion cai para `n-1`.

6. Comparação e significado prático

Busca: $O(n)$ contra $O(\log n)$

Com 10.000 sessões, a busca sequencial faz até 10.000 comparações e a binária, 14. A diferença não é "um pouco mais rápido": é a diferença entre um custo que acompanha o tamanho do arquivo e um custo que praticamente não muda.

Mas a binária tem um preço escondido: ela **exige a lista ordenada**. Se for preciso ordenar antes de cada busca, o custo real é o da ordenação — $O(n^2)$ com os algoritmos desta Sprint — e a busca sequencial sai na frente. A binária compensa quando a lista já está ordenada, ou quando muitas buscas são feitas entre duas alterações. Foi por isso que o programa deixa a escolha na mão do usuário em vez de eleger uma delas.

Ordenação: dois $O(n^2)$ que se comportam diferente

Bubble e insertion pertencem à mesma classe assintótica, e ainda assim o insertion fez, na medição acima, cerca de **metade** das comparações do bubble com entrada aleatória, e **cem vezes menos** com entrada já ordenada.

A lição é sobre o que a notação Big-O diz e o que ela não diz: ela classifica o crescimento no pior caso e ignora constantes. Dois algoritmos $O(n^2)$ podem ter desempenhos muito diferentes no caso médio — e é por isso que a análise assintótica orienta a escolha, mas não substitui a medição.

Tamanho da entrada → operações → desempenho

n	→ operações	→ complexidade	→ desempenho
180	16.110	$O(n^2)$	instantâneo
10.000	~50 milhões	$O(n^2)$	segundos
100.000	~5 bilhões	$O(n^2)$	inviável

Para as 180 sessões do histórico atual, qualquer um dos algoritmos resolve sem que o usuário perceba. Se a rede de eletropostos crescer para dezenas de milhares de sessões, o bubble sort deixa de ser aceitável — e o caminho não é otimizar o laço, é trocar por um algoritmo $O(n \log n)$ como merge sort ou quicksort. **A escolha do algoritmo, e não a velocidade do computador, é o que decide se o sistema escala.**

7. Verificação

O arquivo traz uma bateria de asserções executável:

```
$ python gestao_sesoes.py --autoteste
gestao_sesoes.py OK – busca, ordenação e estatísticas verificadas.
```

Ela confere: busca sequencial encontrando e não encontrando; bubble sort por ID; insertion sort por energia; busca binária sobre lista ordenada, com acerto e com ausência; a contagem de comparações do bubble batendo $n(n-1)/2$; o melhor caso do insertion em $n-1$; os seis valores estatísticos; e a coleção vazia devolvendo zeros sem estourar.

Os mesmos algoritmos também são cobertos pela suíte do projeto (`test_chargenode.py` , classe `TestGestaoSesoes`).